Android akademija

2026

AMA FERIT GDG FERIT teach.ing

ANDROID AKADEMIJA KROZ GODINE

2017 | 2018 | 2024 | 2026

Ovogodišnji predavači:

- Luka Kordić
- Leo Svjetličić
- David Takač
- Ivan Lukac
- Filip Babić
- Marin Tolić
- Martin Zagoršćak
- Goran Luketić
- Tia Ledenčan

Veliko hvala suorganizatoru doc. dr. sc. Bruni Zoriću.

Discord

https://discord.gg/BMPHDqgJxA

GitHub

https://github.com/gdg-osijek/AndroidDevAcademy2026

SADRŽAJ

01 UVOD U KOTLIN

- Kotlin
- Varijable i vrijednosti
- Nullability
- Upravljanje tijekom izvođenja programa
- Petlje
- Funkcije
- Stringovi

02 OOP

- Apstrakcija
- Klase
- Objekti
- Stanje
- Ponašanje
- Enkapsulacija
- Prava pristupa
- Konstruktori
- Object
- Companion object
- Kompozicija

01

Uvod u Kotlin

LET'S HAVE A LOOK

- IntelliJ IDEA
- Kotlin

Kotlin

- Moderan objektno orijentirani jezik
- Tvrtka JetBrains
- Spreman za stvaranje čuda
  - JVM
  - Web
  - Desktop
  - Android
  - Multiplatform
- Kompatibilan s Javom

Varijable i vrijednosti

01

var (*variable*) = varijabla
- klasična varijabla kojoj je moguće mijenjati vrijednost

02

val (*value*) = vrijednost
- varijabla koju je moguće čitati
- vrijednost im može biti pridružena samo jednom
- preferira se njihovo korištenje

03

const (*constant*) = konstanta poznata u vrijeme prevođenja
- primitivnog tipa ili String
- deklaracija na početku **(top-level)** ili unutar **companion object**

Varijable i vrijednosti

- Moguće je napraviti deklaraciju uz naknadnu dodjelu vrijednosti
- Tip nije nužno eksplicitno navoditi već je podržano automatsko određivanje tipa

```
const val PI_VALUE = 3.14159

fun runVariablesExample() {
    val name: String = "Jane Doe"
    var age = 26
    println("Current age: $age")
    age += 1
    println("After one year, $name will be $age years old.")
    println("The value of PI is: $PI_VALUE")
}
```

```
Current age: 26
After one year, Jane Doe will be 27 years old.
The value of PI is: 3.14159
```

Varijable i vrijednosti

Zadatak 1: Napravite varijable za ime igrača, broj života i početne bodove u igri.

- Vaš je igrač osvojio 50 bodova nakon što je pobijedio zlikovca. Ispišite trenutni broj bodova.
- Vaš igrač je pogođen. Izgubio je jedan život. Koliko mu je života preostalo?
- Uvedite dodatnu varijablu za bonus bodove s vrijednosti 2. Pomnožite trenutnu vrijednost bodova s bonus bodovima i ispišite sve vrijednosti varijabli.

Nullability

- Svi tipovi u Kotlinu su takozvani tipovi reference (engl. *reference types*)

= varijabla i vrijednost nisu jedno te isto, varijabla je referenca na stvarnu vrijednost koja se nalazi negdje na hrpi

- Pristup nulabilnoj vrijednosti koristeći **?.** (*safe call operator*)
- Definiranje nulabilnih tipova **Type?**
- Neinicijalizirana referenca ima vrijednost *null*, a pristup članovima objekta preko takve reference dovodi do NPE (*null pointer exception*)

Nullability

- Ako ne definiramo nulabilni tip String , a želimo pridružiti vrijednost null:

```
val nickname: String = null
```

Null can not be a value of a non-null type String

Change type of 'nickname' to 'String?' Alt+Shift+Enter More actions... Alt+Enter

- Znak **?** nakon deklaracije tipa podatka čini tip nulabilnim, odnosno sposobnim poprimiti vrijednost *null*

```
val nickname: String? = null
```

Nullability

```
fun runNullableExample() {
    val name: String = "John"       //non-nullable
    val nickname: String? = null    // nullable

    println(name.length)
    println(nickname?.length)

    //execute code only if it's not null
    nickname?.let { it: String
        println("Nickname defined: $it")
    }
}
```

```
4
null
```

Načini rukovanja null vrijednostima

- Provjera null vrijednosti s **if** uvjetom
- Safe call operator **?.**
- *Elvis operator* ***?:***
- *Not-null assertion operator* ***!!*** -> nije preporučljivo koristiti jer može izazvati NPE
- Funkcija ***let*** i ***?.***
- Sigurna pretvorba s ***as?***

Načini rukovanja null vrijednostima

```kotlin
fun runNullExample() {
    val nickname: String? = null

    val length = if(nickname != null) nickname.length else 0
    val lengthElvis = nickname?.length ?: 0 // Elvis operator ?:

    println(length)
    println(lengthElvis)
    println(nickname!!.length) // not-null assertion operator
}
```

```
0
0
Exception in thread "main" java.lang.NullPointerException Create breakpoint
    at MainKt.runNullExample(Main.kt:382)
    at MainKt.main(Main.kt:5)
```

Pitanja?

Upravljanje tijekom izvođenja programa

Grananje/izraz if-else

- Uvjetno izvođenje koda
- Zadovoljen uvjet? -> izvrši blok naredbi
- Izvršava se samo jedna grana, ostale se ignoriraju
- if-else je izraz u Kotlinu
  - vraća vrijednost koja može biti dodijeljena
  - mora postojati else

Grananje/izraz if-else

```
fun runIfElseExample() {
    var age = 15

    // Traditional usage
    if (age >= 18) {
        println("Adult")
    } else {
        println("Minor")
    }

    //Expression
    age = 20
    val status = if (age >= 18) "Adult" else "Minor"
    println(status)
}
```

```
Minor
Adult
```

Grananje/izraz if-else

Zadatak 2: Vaš prethodno riješeni zadatak proširite koristeći if-else.

- Uvedite varijablu za razinu. Ako igrač ima više od 100 bodova povećat će mu se razina za 2. U slučaju da ima manje od 100 bodova, a više od 50 povećat će mu se razina za 1. U suprotnom, nema promijene razine.
- Napišite izraz u kojemu ćete ispisati „Game Over!” ukoliko igrač nema preostalih života.

When

- Uvjetni izraz
- Nalik naredbi switch
- Sekvencijalna provjera uvjeta svih grana sve dok jedan uvjet nije zadovoljen, ostale se tada ignoriraju
- Ako se koristi kao izraz -> vrijednost izraza = vrijednost prvog zadovoljenog uvjeta

When

```
fun runWhenExample() {
    val day = "Friday"

    val message = when (day) {
        "Monday" -> "Start of the work week!"
        "Friday" -> "Almost weekend!"
        "Saturday", "Sunday" -> "It's the weekend!"
        else -> "Midweek day"
    }

    println(message)
}
```

```
Almost weekend!
```

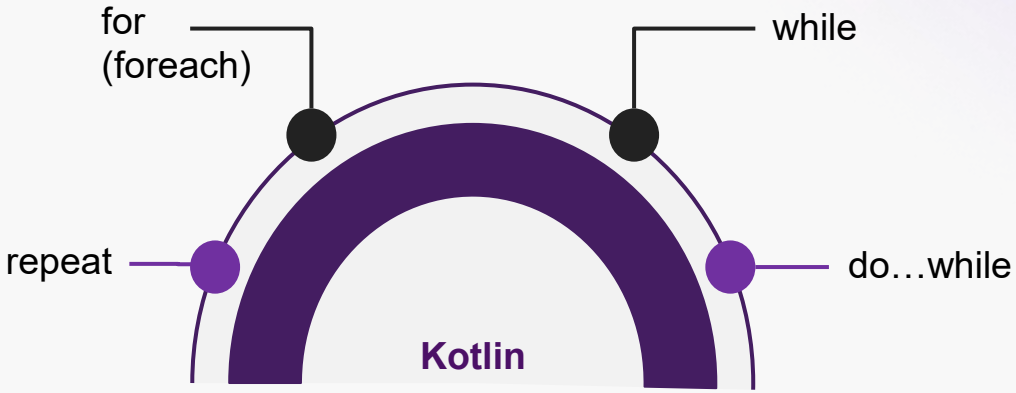
When

- **Zadatak 3:** Koristeći **when** definirajte nekoliko akcija za koje će se igraču povećati bodovi, smanjiti životi i slično. Neka se koristi unos akcije s tipkovnice.

Petlje

Petlje

- Ponavljajuće izvršavanje jedne ili više naredbi do ispunjenja uvjeta



Petlja **for**

- može se rabiti za iteriranje bilo čega što pruža iterator

```
fun runForExample() {
    val fruits = listOf("Apple", "Banana", "Cherry")

    for (fruit in fruits) {
        println(fruit)
    }
    for (i in 1 ≤ .. ≤ 5) {
        println("$i is ${if (i % 2 == 0) "even" else "odd"}")
    }
    for (i in 1 ≤ until < 5) {
        println(i)
    }
    for (i in (10 ≥ downTo ≥ 0 step 2)) {
        println("$i is ${if (i % 2 == 0) "even" else "odd"}")
    }
}
```

```
Apple
Banana
Cherry
```

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
```

```
1
2
3
4
```

```
10 is even
8 is even
6 is even
4 is even
2 is even
0 is even
```

Petlje **while** i **do…while**

- ponavljanje sve dok je kontrolni izraz istinit

```
fun runWhileExample() {
    var i = 1

    while (i <= 5) {
        println(i)
        i++
    }
}
```

```
1
2
3
4
5
```

- uvjet se provjerava nakon svake iteracije
- tijelo petlje se uvijek izvodi barem jednom

```
fun runDoWhileExample() {
    var d = 1

    println("Do while:")
    do {
        println(d)
        d++
    } while (d % 2 == 0)
}
```

```
Do while:
1
2
```

Petlje

Zadatak 4: Napišite program koji uz pomoć for petlje računa i na ekran ispisuje sumu prvih n prirodnih brojeva, gdje se n zadaje s tipkovnice.

Zadatak 5: Izbrojite koliko puta se pojavljuje slovo „a” u predanom stringu koristeći petlju.

Funkcije

- Samostalne, modularne jedinice koda
- Opći oblik:

```
fun ime_funkcije (formalna_lista_parametara): povratni_tip {
        tijelo funkcije – blok naredbi
}
```

- Moguće je rabiti podrazumijevane vrijednosti za argumente
  - Koristi se ako se ne preda argument za taj parametar
- Imenovani argumenti
  - ne mora se poštovati redoslijed definiranih parametara funkcije ako se koriste imenovani argumenti

Funkcije

```kotlin
fun runFunctionsExample() {
    greet( name: "Jane")
    println(multiply( a: 6,  b: 4))
    orderCoffee(sugar = 2, size = "Large")
}

fun greet(name: String) {
    println("Hey $name, welcome to Kotlin!")
}
fun multiply(a: Int, b: Int) = a * b

fun orderCoffee(size: String = "Medium", sugar: Int = 1) {
    println("You ordered a $size coffee with $sugar sugar(s)")
}
```

```
Hey Jane, welcome to Kotlin!
24
You ordered a Large coffee with 2 sugar(s)
```

Funkcije

Zadatak 6: Napišite funkcije koje će služiti za

- dodavanje bodova igraču
- prikaz statusa igrača
- provjeru je li igrač živ
- jednu po vašem izboru

Stringovi

- Nizovi znakova
- Indeksni operator [ ] za pristup elementima
- Stringovi su nepromjenjivi –> izmjena stvara novi string
- Može ih se spajati koristeći
  - operator +
  - interpolacijom stringova

```
fun runStringExample() {
    val name = "John"
    val lastName = "Doe"
    val fullName = name + lastName
    println(fullName)
    val nameWithTemplate = "$name $lastName"
    print(nameWithTemplate)
}
```

```
JohnDoe
John Doe
```

Stringovi

Zadatak 7: Napišite funkciju koja za predani string provjerava bi li on činio dovoljno jaku lozinku. Da bi to činio, mora sadržavati barem jedno veliko slovo, jedan broj te biti dug barem 8 znakova. Testirajte funkciju s nekoliko različitih stringova.

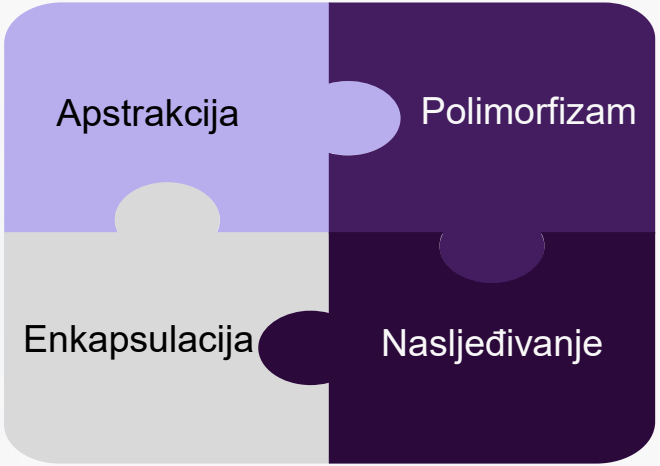
Pitanja?

02

OOP

OOP

Paradigma razvoja



Ideja je prilagoditi programski jezik problemu koji se rješava uvođenjem novih tipova podataka koji grupiraju karakteristike i radnje entiteta uočenih ili u domeni problema ili u domeni rješenja.

Apstrakcija

- Predstavljanje koncepta iz domene problema njegovim karakteristikama i očekivanim radnjama
- Definiranje ponašanja
- Pr. lik u videoigri, kosilica, učitelj

Abstraction: Character
Characteristics: • Name • HP
Responsibilities: • Check if alive • Take damage

Klase

- Korisnički definiran tip podatka
- Stanje (atributi/svojstva) + ponašanje (metode)
- Nacrt prema kojemu se grade objekti
- Definira se ključnom riječi **class** nakon čega se navodi primarni konstruktor i u vitičastim zagradama definira stanje i ponašanje klase

```
class Student

//može se izostaviti constructor jer nema posebno naznačeno pravo pristupa
class Student constructor(val name: String)

class Student(val name: String)
```

Objekti

- Stvarna realizacija klase
- Zauzima memoriju
- Nastaje procesom instanciranja klase -> instanca klase
- Preko varijabli tipa klase (referenci) možemo pristupati objektima i pozivati funkcije/metode na objektu

```
class Student(val name: String, var age: Int)

fun runStudentExample(){
    val student = Student(name = "Jane", age = 21)
    println(student.name)
}
```

Stanje

- Sva svojstva objekta i trenutne vrijednosti tih svojstava
- Vrijednosti mogu biti jednostavne (npr. količina) ili mogu biti kompleksne (npr. drugi objekti)
- Svaki objekt ima vlastito stanje, odnosno vlastita svojstva
- Opći oblik:

```
var propertyName: PropertyType = property_initializer
        access_modifier get = implementation
        access_modifier set = implementation
```

Ponašanje

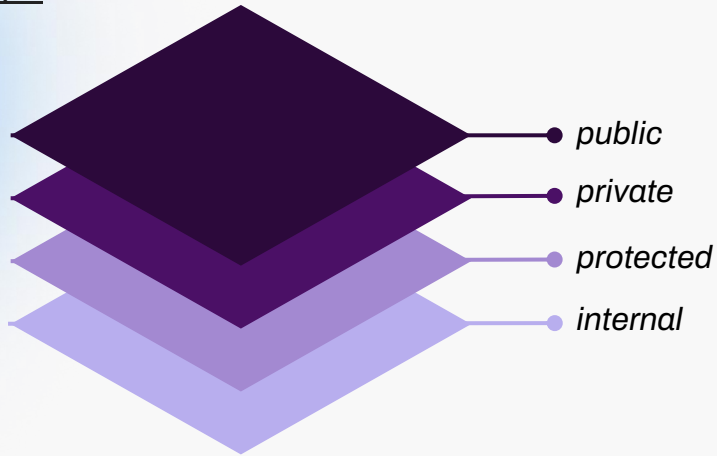
- Način na koji se objekt ponaša i reagira na promjene stanja ili slanja poruka drugim objektima
- Implementira se pomoću funkcija deklariranih unutar klase (metoda)

Enkapsulacija

- Proces grupiranja podataka i metoda u klasu
- Proces razdvajanja javnog sučelja apstrakcije od njegove implementacije pri čemu implementacija mora biti skrivena
- Apstrakcija i enkapsulacija su komplementarne
  - Apstrakcija mora prethoditi enkapsulaciji
  - Da bi apstrakcija bila moguća, implementacija mora biti enkapsulirana
- Najčešće se ostvaruje skrivanjem informacija
  - Skriva se struktura klase, kao i implementacija metoda

Prava pristupa

- Najjednostavniji način za ostvarivanje enkapsulacije
- Ograničavanje pristupa članovima klase
- U Kotlinu 4 tipa



Stanje i ponašanje

```
class Player(
    val name: String,
   initialLives: Int
) {
    //public get, private set
    var score: Int = 0
        private set

    var level: Int = 1
        private set

    var lives: Int = initialLives
    private set

    fun addPoints(points: Int) {
        score += points
        if (score >= 100) {
            levelUp()
        }
    }
    fun takeDamage(damage: Int) {...}

    fun isAlive(): Boolean = lives > 0

    override fun toString(): String {
        return """
            Player: $name
            Lives: $lives
            Level: $level
            Score: $score
        """.trimIndent()
    }
    private fun levelUp() {...}
}
```

```
fun runPlayerExample() {
    val player = Player( name: "John",  lives: 3)
    println(player)
    player.addPoints( points: 220)
    println("An attack")
    if(player.isAlive()){
        player.takeDamage( damage: 60)
    }
    println(player)
}
```

```
Player: John
Lives: 3
Level: 1
Score: 0
John leveled up to level 2!
An attack
Player: John
Lives: 2
Level: 2
Score: 220
```

Konstruktori

- Stvaranje objekta = instanciranje klase
- Posebna metoda klase za stvaranje objekta
- Skupljanje smeća osigurava ispravno rukovanje memorijom
- Kotlin –> primarni i sekundarni konstruktor te blok za inicijalizaciju
- Inicijalizacija u inicijalizacijskom bloku vrijedi za sve konstruktore (***init***)
- Uobičajeno se sekundarni konstruktori pišu kao tzv. delegirani konstruktori
    - posao se delegira primarnom konstruktoru pozivom ***this*** i prosljeđivanjem argumenata odmah nakon formalne liste parametara

Konstruktori

```
class Player(
    val name: String,
    var lives: Int,
    initialScore: Int
) {
    var score: Int = 0
        private set

    var level: Int = 1
        private set

    // INIT block runs after primary constructor
    init {
        println("Creating player: $name")
        if (lives <= 0) {
            println("Lives cannot be 0 or negative. Setting to 1.")
            lives = 1
        }
        score = if (initialScore >= 0) initialScore else 0
    }
    // Secondary constructor
    constructor(name: String) : this(name, lives: 3, initialScore: 0) {
        println("Secondary constructor used (default player).")
    }

    fun addPoints(points: Int) {...}

    override fun toString(): String {
        return "Player(name='$name', lives=$lives, level=$level, score=$score)\n"
    }
}
```

Konstruktori

```
fun runConstructorExample() {
    println("---- Primary Constructor ----")
    val player1 = Player( name: "Alex",  lives: 5,  initialScore: 20)
    println(player1)

    println("---- Secondary Constructor ----")
    val player2 = Player( name: "Bob")
    println(player2 )

    println("---- Testing Validation ----")
    val player3 = Player( name: "Eve",  lives: -2,  initialScore: -50)
    println(player3)
}
```

```
---- Primary Constructor ----
Creating player: Alex
Player(name='Alex', lives=5, level=1, score=20)

---- Secondary Constructor ----
Creating player: Bob
Secondary constructor used (default player).
Player(name='Bob', lives=3, level=1, score=0)

---- Testing Validation ----
Creating player: Eve
Lives cannot be 0 or negative. Setting to 1.
Player(name='Eve', lives=1, level=1, score=0)
```

Konstruktori

- **Zadatak 8:** Izradite klasu koja modelira sustav korisničkih računa (*UserAccount*)

Klasa treba imati primarni konstruktor s korisničkim imenom, email adresom, dobi
Dodajte init blok koji baca iznimku ako je dob < 13 te ispisuje "Korisnik je kreiran: <korisničko_ime>„.
Dodajte svojstvo *isVerified* (zadano false).
Dodajte sekundarni konstruktor koji omogućuje stvaranje korisnika samo sa korisničkim imenom i email adresom. U ovom slučaju: dob bi trebala biti zadana vrijednost 18.
Dodajte metodu *verifyEmail* koja postavlja *isVerified* = true.

U main() stvorite tri korisnika koristeći različite konstruktore.

Bonus: Pokušajte dodati
- Izračunatu vrijednost *isAdult.*
- Validaciju konstruktora koja provjerava sadrži li email adresa "@".
- Sekundarni konstruktor koji uzima jedan niz poput "ime:email:dob" i parsira ga.

Object

- Obrazac *Singleton*
- Potreba postojanja jedne instance neke klase
- Ključna riječ ***object*** – kreira se jedinstveni objekt (kreira se klasa i instancira objekt u istom koraku)
- Objektu se pristupa preko imena, ne instancira se klasa

Object

```
object Logger {
    var isEnabled: Boolean = true

    fun printLog(message: String) {
        if(isEnabled) {
            println("[LOG] $message")
        }
    }
}
```

```
fun runObjectExample() {
    Logger.printLog("The application has started.")
    Logger.isEnabled = false
    Logger.printLog("Show log message.")
}
```

```
[LOG] The application has started.
```

Companion object

- Varijable i metode instance predstavljaju stanje i ponašanje nekog objekta
- Često se javlja potreba da se varijable i metode definiraju neovisno o instanci, u drugim jezicima to se postiže korištenjem ključne riječi static
- Kotlin ima na raspolaganju takozvane companion objekte
- Moguće im je pristupati eksplicitno ili implicitno

Companion object

```
class Player(val name: String) {

    var score = 0
        private set

    fun addPoints(points: Int) {
        score += points
        updateHighScore( player: this)
    }

    companion object {

        var highScore = 0
            private set

        var bestPlayer: String = "None"
            private set

        private fun updateHighScore(player: Player) {
            if (player.score > highScore) {
                highScore = player.score
                bestPlayer = player.name
            }
        }

        fun showBestHighScore() {
            println("High Score: $highScore by $bestPlayer")
        }
    }
}
```

```
fun runCompanionObjectExample() {
    val p1 = Player( name: "Alex")
    val p2 = Player( name: "Bob")

    p1.addPoints( points: 50)
    p2.addPoints( points: 100)

    Player.showBestHighScore()
}
```

High Score: 100 by Bob

Kompozicija

- Svaka klasa treba imati samo jednu odgovornost
- Složena ponašanja ostvaruju se suradnjom objekata više klasa
- Klase kao svojstva mogu imati objekte drugih klasa
- Kompozicija opisuje odnos **has-a**
- Automobil ima motor, škola ima učionice, Android akademija ima predavače

Kompozicija

```
class Engine(val horsePower: Int) {
    fun start() {
        println("Vroom! $horsePower HP engine is running.")
    }
}

class Car(private val engine: Engine) {

    fun drive() {
        engine.start()
        println("The car is moving!")
    }
}
```

Kompozicija

```kotlin
fun runCompositionExample() {
    val v8Engine = Engine(horsePower = 450)
    val ecoEngine = Engine(horsePower = 120)

    val sportsCar = Car(v8Engine)
    val cityCar = Car(ecoEngine)

    println("Sports car")
    sportsCar.drive()

    println("City car")
    cityCar.drive()
}
```

```
Sports car
Vroom! 450 HP engine is running.
The car is moving!
City car
Vroom! 120 HP engine is running.
The car is moving!
```

THANK YOU

DO you have any questions?

CREDITS: This presentation template was created by **Slidesgo**, and includes icons, infographics & images by **Freepik**

SLIDESGO